

RELATÓRIO PARTE 2 — Transmissão para Nuvem e Visualização (Fog/Cloud Computing)

Grupo: Ana Cornachi e Carla Máximo

1. Visão Geral da Arquitetura

A Parte 2 implementa a camada de transmissão e visualização do Cardiola, integrando o dispositivo de borda (Edge) com a nuvem por meio do protocolo MQTT e visualizando os dados em tempo real através de um dashboard no Node-RED.

A arquitetura segue o modelo de três camadas amplamente adotado em projetos de IoT para saúde:

- **Edge:** ESP32 (simulado via script Python) captura e publica os sinais vitais diretamente no broker, representando o dispositivo vestível do paciente
- **Fog:** Broker HiveMQ Cloud recebe, autentica e roteia as mensagens MQTT entre os dispositivos publicadores e assinantes
- **Cloud:** Node-RED consome as mensagens do broker, processa os dados e exibe em dashboard de monitoramento em tempo real

Essa separação em camadas é fundamental para garantir escalabilidade, em um cenário real, múltiplos pacientes poderiam publicar dados simultaneamente no mesmo broker, com o Node-RED consumindo e exibindo cada fluxo de forma independente.

2. Protocolo MQTT — Conceito e Funcionamento

O MQTT (Message Queuing Telemetry Transport) é um protocolo de comunicação leve e eficiente, criado originalmente para telemetria em ambientes com largura de banda limitada e alta latência, como redes de satélite. Por essas características, tornou-se o protocolo padrão para comunicação em dispositivos IoT com recursos limitados como o ESP32.

O MQTT opera no modelo publish/subscribe, diferente do modelo request/response tradicional (como HTTP). Nesse modelo:

- O **publisher** (ESP32/Python) publica mensagens em um tópico sem precisar saber quem vai receber
- O **broker** (HiveMQ) recebe e distribui as mensagens para todos os assinantes do tópico

- O **subscriber** (Node-RED) assina o tópico e recebe automaticamente todas as mensagens publicadas

Essa arquitetura desacoplada é ideal para monitoramento médico: o dispositivo vestível do paciente não precisa conhecer o sistema de visualização, e múltiplos dashboards podem assinar o mesmo tópico simultaneamente — permitindo que médicos, enfermeiros e sistemas de alertas recebam os mesmos dados em paralelo.

3. Configuração do Broker HiveMQ Cloud

Para este projeto foi utilizado o HiveMQ Cloud como broker MQTT na nuvem, com as seguintes configurações técnicas:

- **Broker:** `dc8d802607554c6cb32cacfed071c8b9.s1.eu.hivemq.cloud`
- **Porta:** 8883 (TLS/SSL — comunicação criptografada)
- **Tópico:** `cardioia/sinais`
- **QoS:** 1 (garantia de entrega ao menos uma vez)
- **Autenticação:** usuário e senha com permissão Publish and Subscribe

A escolha da porta 8883 com TLS é fundamental para projetos de IoT médico. Diferente da porta 1883 (sem criptografia), a 8883 garante que os dados de saúde dos pacientes trafeguem de forma criptografada entre o dispositivo e a nuvem, protegendo informações sensíveis contra interceptação. Essa prática está alinhada com a LGPD (Lei Geral de Proteção de Dados) e com as diretrizes de segurança para sistemas de saúde digitais.

O QoS 1 foi escolhido como equilíbrio entre confiabilidade e performance: garante que cada mensagem seja entregue ao menos uma vez, sem o overhead do QoS 2 (exatamente uma vez), adequado para sinais vitais onde uma pequena duplicação é aceitável mas a perda de dados não é.

O publisher foi implementado em Python (`src/mqtt/publisher.py`) simulando o ESP32, publicando a cada 3 segundos um payload JSON com os sinais vitais:

```
{"temp": 38.2, "umid": 65.3, "bpm": 95, "ts": 1715456789}
```

O campo `ts` (timestamp Unix) permite rastrear exatamente quando cada leitura foi realizada, essencial para análise histórica e auditoria dos dados clínicos.

4. Configuração do Dashboard no Node-RED

O Node-RED foi utilizado como plataforma de visualização Fog/Cloud, consumindo as mensagens do broker HiveMQ e exibindo os dados em tempo real. A biblioteca

utilizada foi o @flowfuse/node-red-dashboard (Dashboard 2.0), compatível com Node-RED v4.

O flow foi estruturado da seguinte forma:

- **Nó MQTT In:** assina o tópico cardioia/sinais e recebe os payloads JSON do broker
- **Nós Function (x4):** extraem individualmente cada sinal vital (temperatura, umidade, BPM) do payload e encaminham para os componentes visuais correspondentes. O quarto nó avalia os limites clínicos e gera o texto de alerta
- **Gauges (x3):** exibem o valor atual de temperatura, umidade e BPM em medidores visuais semicirculares com escala configurada clinicamente
- **Charts (x2):** exibem o histórico de temperatura e BPM ao longo do tempo em gráficos de linha, permitindo identificar tendências e variações
- **Text (x1):** exibe o status de alerta atual de forma textual e destacada

Os dados fluem do broker para o dashboard em menos de 1 segundo, garantindo monitoramento praticamente em tempo real — fundamental para detecção precoce de eventos cardíacos críticos.

5. Limites de Alerta Definidos

Os limites foram definidos com base em referências clínicas cardiológicas amplamente utilizadas na medicina:

Sinal	Condição	Limite	Alerta
Temperatura	Febre	> 38°C	⚠️ Febre
BPM	Taquicardia	> 120 bpm	⚠️ Taquicardia
BPM	Bradicardia	< 40 bpm	⚠️ Bradicardia
Umidade	Umidade elevada	> 80%	⚠️ Umidade alta

Quando nenhum limite é ultrapassado, o dashboard exibe  **Sinais vitais normais**, indicando ao profissional de saúde que o paciente está estável.

A lógica de alertas foi implementada no Node-RED via nó function, avaliando cada leitura recebida em tempo real. Em um sistema de produção, esses alertas poderiam ser expandidos para disparar notificações push, emails ou SMS para a equipe médica responsável.

6. Segurança e Boas Práticas em IoT Médico

A segurança dos dados é um aspecto crítico em qualquer sistema de IoT aplicado à saúde. Neste projeto, as seguintes boas práticas foram adotadas:

Criptografia em trânsito: toda a comunicação entre o publisher e o broker HiveMQ utiliza TLS na porta 8883, garantindo que os dados de saúde não possam ser interceptados por terceiros durante a transmissão.

Autenticação: o acesso ao broker é protegido por usuário e senha, evitando que dispositivos não autorizados publiquem ou consumam dados no tópico `cardioia/sinais`.

Dados simulados: nenhum dado real de paciente foi utilizado neste projeto. Todos os sinais vitais são gerados de forma aleatória dentro de faixas clinicamente plausíveis, eliminando riscos de exposição de dados pessoais de saúde.

Separação de responsabilidades: a arquitetura em camadas (Edge → Fog → Cloud) garante que cada componente tem uma responsabilidade específica e bem definida, facilitando a manutenção, auditoria e evolução do sistema.

7. Importância para IoT Médico e Reflexões Finais

A integração MQTT + Node-RED demonstra na prática como dados de dispositivos vestíveis podem ser transmitidos, processados e visualizados em tempo real com baixa latência e alta confiabilidade. Em um cenário real de mercado, esse pipeline permitiria que profissionais de saúde monitorassem remotamente pacientes cardíacos, recebendo alertas automáticos em situações de risco sem necessidade de presença física.

Os benefícios diretos dessa arquitetura para a saúde cardiovascular incluem: detecção precoce de arritmias e episódios de taquicardia, monitoramento contínuo de pacientes em recuperação pós-cirúrgica, redução de internações desnecessárias por meio do acompanhamento domiciliar, e maior agilidade na tomada de decisão clínica com dados em tempo real.

Do ponto de vista de governança de dados — tema central do projeto Cardiola desde a Fase 1 — este sistema reforça que a coleta e transmissão de sinais vitais deve ser feita com responsabilidade: dados criptografados, acesso controlado, sem exposição de informações pessoais e com rastreabilidade via timestamp em cada leitura.

8. Evidências do Sistema em Funcionamento



Figura 1 — Dashboard Cardiola exibindo sinais vitais dentro dos limites normais

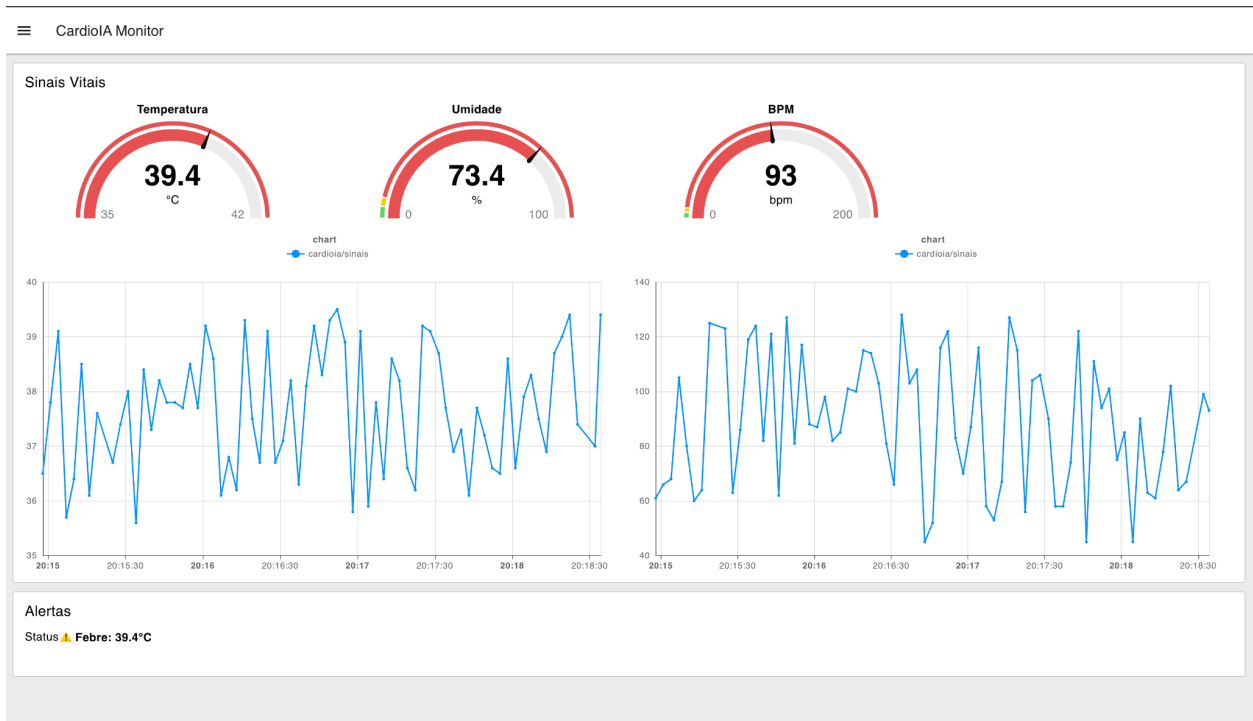


Figura 2 — Dashboard CardiolA exibindo alerta de febre com temperatura de 39.4°C

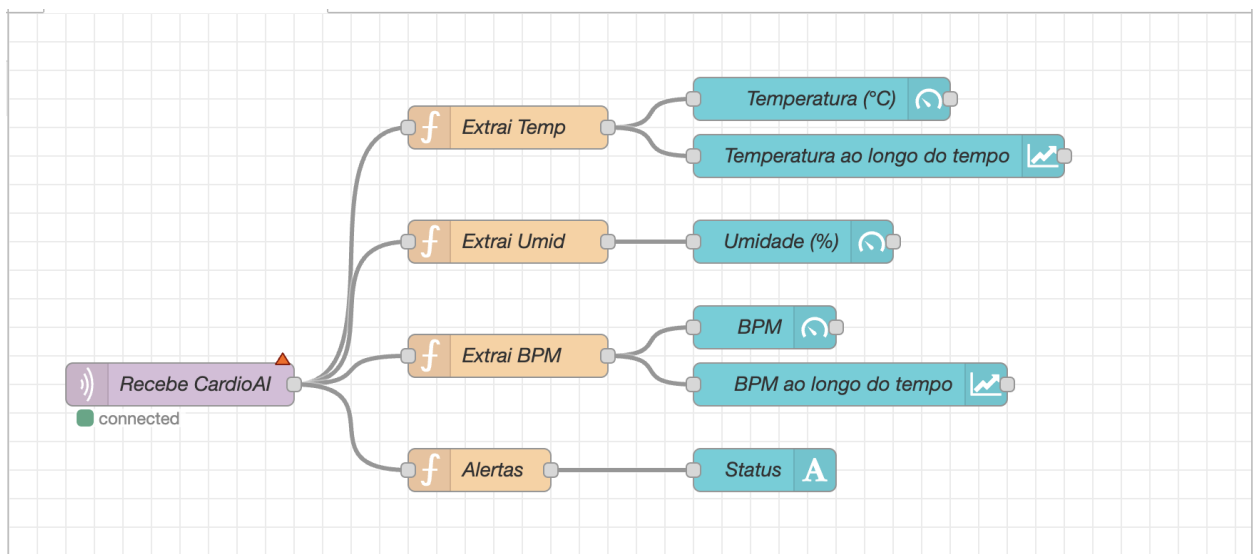


Figura 3 — Flow completo no Node-RED com nós MQTT, functions, gauges e charts

```
⚠️ ALERTA: Temperatura elevada (febre)!  
⚠️ ALERTA: BPM elevado (taquicardia)!  
✉️ Mensagem 431 publicada com sucesso  
🌡️ Temp: 38.9°C 💧 Umid: 54.4% ❤️ BPM: 128  
⚠️ ALERTA: Temperatura elevada (febre)!  
⚠️ ALERTA: BPM elevado (taquicardia)!  
✉️ Mensagem 432 publicada com sucesso  
🌡️ Temp: 36.9°C 💧 Umid: 73.2% ❤️ BPM: 69  
✉️ Mensagem 433 publicada com sucesso  
🌡️ Temp: 38.9°C 💧 Umid: 61.6% ❤️ BPM: 99  
⚠️ ALERTA: Temperatura elevada (febre)!  
✉️ Mensagem 434 publicada com sucesso  
🌡️ Temp: 38.6°C 💧 Umid: 57.7% ❤️ BPM: 49  
⚠️ ALERTA: Temperatura elevada (febre)!  
✉️ Mensagem 435 publicada com sucesso  
🌡️ Temp: 38.8°C 💧 Umid: 40.7% ❤️ BPM: 66  
⚠️ ALERTA: Temperatura elevada (febre)!  
✉️ Mensagem 436 publicada com sucesso  
🌡️ Temp: 35.8°C 💧 Umid: 73.6% ❤️ BPM: 45  
✉️ Mensagem 437 publicada com sucesso
```

```
□
```

Figura 4 — Publisher Python simulando o ESP32 e publicando dados no broker HiveMQ